Taller 1: Teoría de Aprendizaje de Máquina – 2026-2

Clasificación supervisada, vecinos cercanos y evaluación de modelos

Cristian David Dorado Gómez

Profesor: Diego Armando Pérez Rosero

8 de noviembre de 2026

Resumen

Se resuelven los tres puntos del taller: (1) clasificación lineal con el perceptrón, incluyendo la frontera de decisión, la demostración de crecimiento del margen y un ejercicio numérico; (2) clasificación por k vecinos más cercanos, con el cálculo manual de distancias y votación, y la evaluación experimental sobre `make_moons` ($N = 300$, `noise=0.20`, `random_state=42`); y (3) evaluación de un clasificador binario con matriz de confusión, métricas manuales y verificación en `scikit-learn`. Todos los resultados numéricos de este documento provienen de la ejecución del cuaderno reproducible `solucion_taller.ipynb`, con semillas fijas (`np.random.seed(42)`).

Índice

1. Punto 1: clasificación lineal y algoritmo del perceptrón **1**
- 1.1. Frontera de decisión . . . 2
- 1.2. Actualización del perceptrón y el margen . . . 3
- 1.3. Ejercicio numérico . . . 3

2. Punto 2: clasificación por los k vecinos más cercanos **4**
- 2.1. Cálculo manual de distancias y votación . . . 4
- 2.2. Implementación sobre `make_moons` . . . 5
- 2.3. Interpretación . . . 6

3. Punto 3: evaluación de un clasificador binario **7**
- 3.1. Matriz de confusión . . . 7
- 3.2. Métricas . . . 8
- 3.3. Verificación en Python y análisis . . . 8

4. Conclusiones **9**

Reproducibilidad **10**

A. Código de las partes implementadas manualmente **10**

1. Punto 1: clasificación lineal y algoritmo del perceptrón

El clasificador binario asigna

$$\hat{y} = \begin{cases} +1, & \mathbf{w}^\top \mathbf{x} + b > 0, \\ -1, & \mathbf{w}^\top \mathbf{x} + b \leq 0, \end{cases} \qquad \mathbf{w} \in \mathbb{R}^2,\ b \in \mathbb{R}, \tag{1}$$

es decir, $\hat{y} = \text{sign}(\mathbf{w}^\top \mathbf{x} + b)$ con la convención de que el valor 0 se asigna a la clase -1. La función $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$ es el *puntaje* o función de decisión: su signo da la clase y su magnitud (absoluta, dividida por $\|\mathbf{w}\|$) la distancia a la frontera.

1.1. Frontera de decisión

Con $\mathbf{w} = (1, 2)^\top$ y $b = -3$ la frontera $\mathbf{w}^\top \mathbf{x} + b = 0$ es

$$x_1 + 2x_2 - 3 = 0 \quad \Longleftrightarrow \quad x_2 = \frac{3 - x_1}{2} = 1.5 - 0.5\, x_1, \tag{2}$$

una recta de pendiente $-1/2$ que corta el eje x_1 en $x_1 = 3$ y el eje x_2 en $x_2 = 1.5$. El vector $\mathbf{w} = (1, 2)^\top$ es *normal* a la recta y apunta hacia el semipiano de la clase $+1$: la región $x_1 + 2x_2 > 3$ es la clase $+1$ y $x_1 + 2x_2 < 3$ es la clase -1; la recta misma pertenece a -1 por la convención ≤ 0. La Figura 1 muestra la recta y los cuatro puntos.

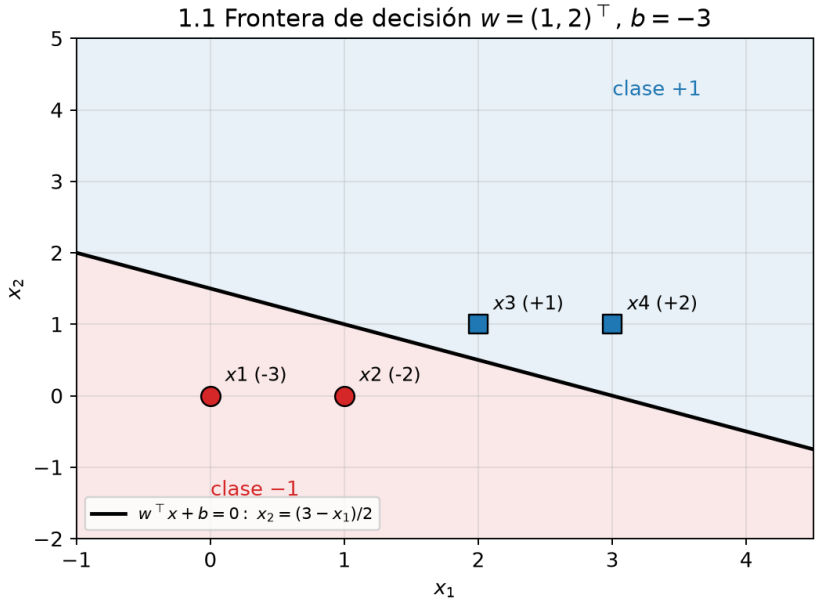


Figura 1: 1.1 Frontera de decisión para $\mathbf{w} = (1, 2)^\top$, $b = -3$ y clasificación de $x_1, \ldots, x_4$. El valor entre paréntesis es $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$.

Sustituyendo cada punto en $f(\mathbf{x}) = x_1 + 2x_2 - 3$ se obtiene la Tabla 1. La justificación de cada predicción es simplemente el signo de f: no se requiere otra regla.

Cuadro 1: 1.1 Evaluación de la regla de decisión en los cuatro puntos.

Observación	$\mathbf{x}$	$f(\mathbf{x}) = x_1 + 2x_2 - 3$	Signo	$\hat{y}$
x_1	$(0, 0)$	$0 + 0 - 3 = -3$	< 0	-1
x_2	$(1, 0)$	$1 + 0 - 3 = -2$	< 0	-1
x_3	$(2, 1)$	$2 + 2 - 3 = +1$	> 0	$+1$
x_4	$(3, 1)$	$3 + 2 - 3 = +2$	> 0	$+1$

Las distancias a la frontera son $|f(\mathbf{x})|/\|\mathbf{w}\|$ con $\|\mathbf{w}\| = \sqrt{5} \approx 2.2361$: 1.3416 y 0.8944 para x_1 y x_2 (lado negativo), y 0.4472 y 0.8944 para x_3 y x_4 (lado positivo). Los dos puntos negativos están más lejos de la frontera que x_3, que es el punto positivo más cercano a ella.

1.2. Actualización del perceptrón y el margen

Con la regla

$$\mathbf{w}_{\text{nuevo}} = \mathbf{w} + \eta\, y_i\, \mathbf{x}_i, \qquad b_{\text{nuevo}} = b + \eta\, y_i, \qquad \eta > 0, \tag{3}$$

y el margen $m_i = y_i\,(\mathbf{w}^\top \mathbf{x}_i + b)$, el margen de la misma observación inmediatamente después de la actualización es

$$\begin{aligned} m_{i,\text{nuevo}} &= y_i \left(\mathbf{w}_{\text{nuevo}}^\top \mathbf{x}_i + b_{\text{nuevo}}\right) \\ &= y_i \left((\mathbf{w} + \eta\, y_i\, \mathbf{x}_i)^\top \mathbf{x}_i + b + \eta\, y_i\right) \\ &= y_i \left(\mathbf{w}^\top \mathbf{x}_i + b\right) + \eta\, y_i\, (y_i\, \mathbf{x}_i^\top \mathbf{x}_i) + \eta\, y_i^2 \\ &= m_i + \eta\, y_i^2 \|\mathbf{x}_i\|_2^2 + \eta\, y_i^2. \end{aligned} \tag{4}$$

Como $y_i \in \{-1, +1\}$, se tiene $y_i^2 = 1$ y por lo tanto

$$\boxed{m_{i,\text{nuevo}} = m_i + \eta \left(\|\mathbf{x}_i\|_2^2 + 1\right)} \qquad \text{con } \eta \left(\|\mathbf{x}_i\|_2^2 + 1\right) > 0. \tag{5}$$

Qué implica. El incremento es *estrictamente positivo* y *no depende del valor previo de* m_i: sin importar cuán mal clasificada esté la observación, la actualización la desplaza siempre hacia el lado correcto de la frontera en una cantidad fija (que crece con $\|\mathbf{x}_i\|^2$). Dicho de otro modo, el error cometido sobre esa observación se corrige en la dirección correcta y el perceptrón «refuerza» la etiqueta verdadera y_i.

Dos precisiones que conviene explicitar:

- El margen *geométrico* es $m_i/\|\mathbf{w}\|$. El numerador crece, pero $\|\mathbf{w}\|$ también cambia con cada actualización, de modo que el margen geométrico no es necesariamente monótono; la fórmula garantiza el crecimiento del margen funcional.
- Una sola actualización no basta para que un punto muy mal clasificado quede bien clasificado: el desplazamiento es fijo, mientras que $-\eta m_i$ puede ser mucho mayor. Lo que garantiza la fórmula es el avance constante, y es la base de la cota de convergencia del perceptrón: si los datos son linealmente separables con margen geométrico γ y $R = \max_i \|\mathbf{x}_i\|$, el número de errores está acotado por $(R/\gamma)^2$.

Verificación numérica (celda de código del cuaderno, cinco casos aleatorios con error forzado): en todos se cumple $|m_{i,\text{nuevo}} - (m_i + \eta(\|\mathbf{x}_i\|^2 + 1))| < 10^{-12}$ y $m_{i,\text{nuevo}} > m_i$; por ejemplo, para un caso con $m_i = -1.5102$ y $\eta = 0.5$ el margen nuevo es 1.7409, exactamente $m_i + \eta(\|\mathbf{x}_i\|^2 + 1)$.

1.3. Ejercicio numérico

Datos: $\mathbf{w} = (0, 0)^\top$, $b = 0$, $\eta = 0.5$, $\mathbf{x} = (2, 1)^\top$, $y = +1$.

Paso 1: ¿hay error? El puntaje es $f(\mathbf{x}) = 0 \cdot 2 + 0 \cdot 1 + 0 = 0$. La regla de predicción asigna $+1$ sólo si $f(\mathbf{x}) > 0$; como 0 no es mayor que cero, $\hat{y} = -1$. Con $y = +1$ y $\hat{y} = -1$ la observación está mal clasificada (margen $m = y\, f(\mathbf{x}) = 0 \leq 0$); **sí corresponde actualizar**.

Paso 2: actualización con $\eta\, y = 0.5 \cdot (+1) = 0.5$:

$$\mathbf{w}_{\text{nuevo}} = \begin{pmatrix} 0 \\ 0 \end{pmatrix} + 0.5\,(+1) \begin{pmatrix} 2 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \\ 0.5 \end{pmatrix}, \qquad b_{\text{nuevo}} = 0 + 0.5\,(+1) = 0.5. \tag{6}$$

Paso 3: nuevo margen

$$m_{\text{nuevo}} = y\left(\mathbf{w}_{\text{nuevo}}^{\top}\mathbf{x} + b_{\text{nuevo}}\right) = (+1)\,(1\cdot 2 + 0.5\cdot 1 + 0.5) = 3. \tag{7}$$

Comprobación con la fórmula de 1.2: $m + \eta(\|\mathbf{x}\|_2^2 + 1) = 0 + 0.5\,(4 + 1 + 1) = 3$. ✓
El nuevo puntaje es $+3 > 0$, así que la observación queda bien clasificada, con margen funcional 3 y margen geométrico $3/\|\mathbf{w}_{\text{nuevo}}\| = 3/\sqrt{1.25} \approx 2.6833$.

2. Punto 2: clasificación por los k vecinos más cercanos

Observación nueva $q = (1, 1)$ y conjunto de entrenamiento

Observación	x_1	x_2	x_3	x_4	x_5
$\mathbf{x}_i$	$(0, 0)$	$(1, 0)$	$(3, 1)$	$(0, 3)$	$(1.1,\ 1.1)$
Clase	A	A	B	B	B

2.1. Cálculo manual de distancias y votación

Con la distancia euclidiana $d(q, \mathbf{x}_i) = \sqrt{(q_1 - x_{i,1})^2 + (q_2 - x_{i,2})^2}$:

$$\begin{aligned}
d(q, \mathbf{x}_1) &= \sqrt{(1-0)^2 + (1-0)^2} = \sqrt{2} \approx 1.4142,\\
d(q, \mathbf{x}_2) &= \sqrt{(1-1)^2 + (1-0)^2} = \sqrt{1} = 1.0000,\\
d(q, \mathbf{x}_3) &= \sqrt{(1-3)^2 + (1-1)^2} = \sqrt{4} = 2.0000,\\
d(q, \mathbf{x}_4) &= \sqrt{(1-0)^2 + (1-3)^2} = \sqrt{5} \approx 2.2361,\\
d(q, \mathbf{x}_5) &= \sqrt{(1-1.1)^2 + (1-1.1)^2} = \sqrt{0.02} \approx 0.1414.
\end{aligned}$$

Todas las distancias son distintas, de modo que el orden es único y no hay empates que romper (Tabla 2, Figura 2).

Cuadro 2: 2.1 Orden de cercanía a $q = (1, 1)$ y votación mayoritaria.

Puesto	Observación	Distancia	Clase
1	$x_5 = (1.1, 1.1)$	0.1414	B
2	$x_2 = (1, 0)$	1.0000	A
3	$x_1 = (0, 0)$	1.4142	A
4	$x_3 = (3, 1)$	2.0000	B
5	$x_4 = (0, 3)$	2.2361	B
$k = 1$: $\{x_5\} \to$ A: 0, B: 1 $\to \hat{y} =$ **B**			
$k = 3$: $\{x_5, x_2, x_1\} \to$ A: 2, B: 1 $\to \hat{y} =$ **A**			
$k = 5$: todos $\to$ A: 2, B: 3 $\to \hat{y} =$ **B**			

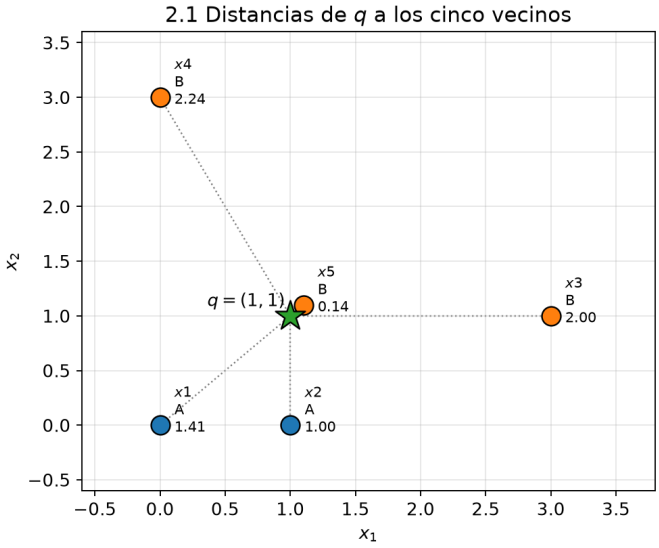


Figura 2: 2.1 Distancias de q a los cinco vecinos (la etiqueta bajo cada punto indica clase y distancia).

Por qué k puede cambiar la predicción. El vecino más cercano, x_5, está a 0.1414 de q y es de clase B: un punto casi pegado a q. Con $k = 3$ entran x_2 y x_1, ambos A, que son mayoría y anulan el voto de x_5; con $k = 5$ se incorporan x_3 y x_4, ambos B, y la mayoría (3 a 2) depende ahora de puntos a más de dos unidades de q, es decir, de la tendencia global del conjunto y no de la vecindad local de q. En general, k controla el compromiso sesgo–varianza: k pequeño usa sólo la vecindad inmediata (fronteras irregulares, muy sensibles al ruido); k grande promedia sobre una región amplia (fronteras suaves, pero puede diluir la estructura local y arrastrar la clase mayoritaria). Aquí, con $N = 5$ y sin empates, la predicción «oscila» B → A → B.

2.2. Implementación sobre `make_moons`

Protocolo. $N = 300$ observaciones bidimensionales generadas con `make_moons(n_samples=300, noise=0.20, random_state=42)`; partición 70 % entrenamiento (210 observaciones) y 30 % prueba (90), estratificada con `random_state=42`, de modo que cada subconjunto conserva 50 %/50 % de clases; `KNeighborsClassifier` con $k \in \{1, 5, 15\}$, distancia euclidiana y votación uniforme (valores por defecto). Se reporta exactitud (*accuracy*) en entrenamiento y en prueba (Tabla 3), y se dibujan los datos (Figura 3) y las regiones de decisión para $k = 1$ y $k = 15$ (Figura 4).

Cuadro 3: 2.2 Exactitud de k-NN sobre `make_moons` (210 entrenamiento / 90 prueba).

k	Exactitud entrenamiento	Exactitud prueba
1	1.0000	0.9333
5	0.9571	**0.9556**
15	0.9571	0.9333

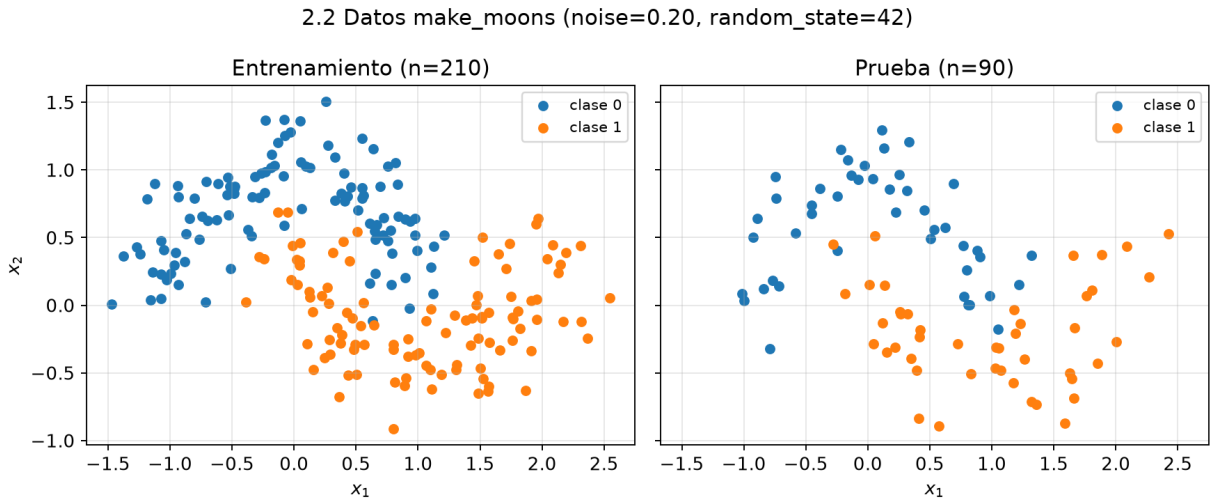


Figura 3: 2.2 Datos de `make_moons` separados en entrenamiento y prueba.

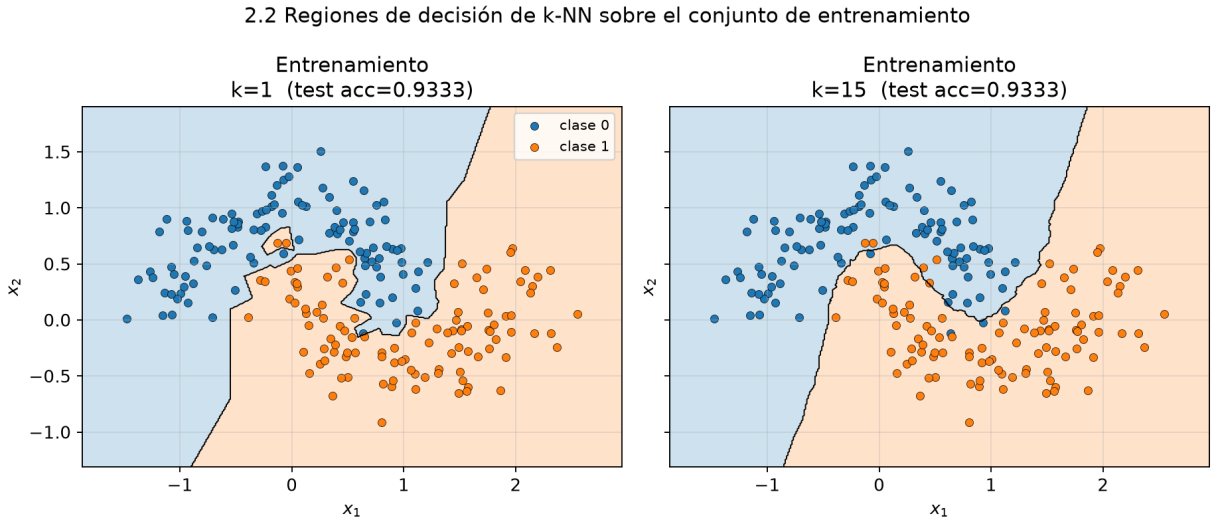


Figura 4: 2.2 Regiones de decisión para $k = 1$ (izquierda) y $k = 15$ (derecha) sobre el conjunto de entrenamiento. Con $k = 1$ la frontera se ajusta a cada punto (incluidos los ruidosos); con $k = 15$ es más suave.

2.3. Interpretación

k muy pequeño. Con $k = 1$ cada observación se clasifica según su vecino inmediato, así que la exactitud de entrenamiento es 1.0000 por construcción (cada punto es su propio vecino más cercano). Es un modelo de *varianza alta*: la frontera es muy irregular, se amolda al ruido (Figura 4, izquierda) y pequeñas perturbaciones de los datos cambian mucho la predicción; pueden aparecer islas de una clase dentro de la otra.

k muy grande. Al promediar muchos vecinos, la frontera se suaviza y se vuelve estable, pero el sesgo aumenta: si k es comparable con el tamaño de la muestra, la vecindad cubre casi todo el espacio y la predicción tiende a la clase mayoritaria, perdiendo la estructura de las dos lunas. Con $k = 15$ ya se observa una frontera notoriamente más limpia.

Cuál generalizó mejor. Las exactitudes de prueba son 0.9333 ($k = 1$), 0.9556 ($k = 5$) y 0.9333 ($k = 15$): el mejor de los tres es $k = 5$ (86/90 aciertos). La justificación no es la exactitud de

entrenamiento —que siempre favorece a $k = 1$— sino el desempeño sobre datos no vistos: $k = 1$ sufre el ruido de los puntos de la frontera entre las lunas y $k = 15$ suaviza tanto que pierde puntos de las «colas»; $k = 5$ conserva la estructura local y descarta el ruido puntual. Las diferencias son pequeñas (una o dos observaciones de prueba), lo que indica que con `noise=0.20` el problema es relativamente sencillo para k-NN.

Por qué k debe elegirse con validación y no con prueba. La exactitud de prueba estima el desempeño de un modelo *ya elegido* sobre datos que el procedimiento no utilizó. Si se prueban muchos valores de k y se escoge el que maximiza la exactitud de prueba, ese conjunto deja de ser una muestra independiente: el número reportado queda sesgado de forma optimista y sobreestima el desempeño real (fuga de información desde el conjunto de evaluación hacia la selección del modelo). El procedimiento correcto es seleccionar k *dentro* del conjunto de entrenamiento con validación cruzada estratificada (o con un conjunto de validación aparte) y reservar el conjunto de prueba para un único reporte final. La Figura 5 muestra esa selección con 5 pliegues sobre las 210 observaciones de entrenamiento, para k impar entre 1 y 39: el máximo de la exactitud media de validación se alcanza en $k = 11$ (0.9619), y su exactitud de prueba, reportada una sola vez, es 0.9667. Nótese que esta cifra no se usó para elegir k: si se hubiera hecho así, el valor reportado habría sido optimista.

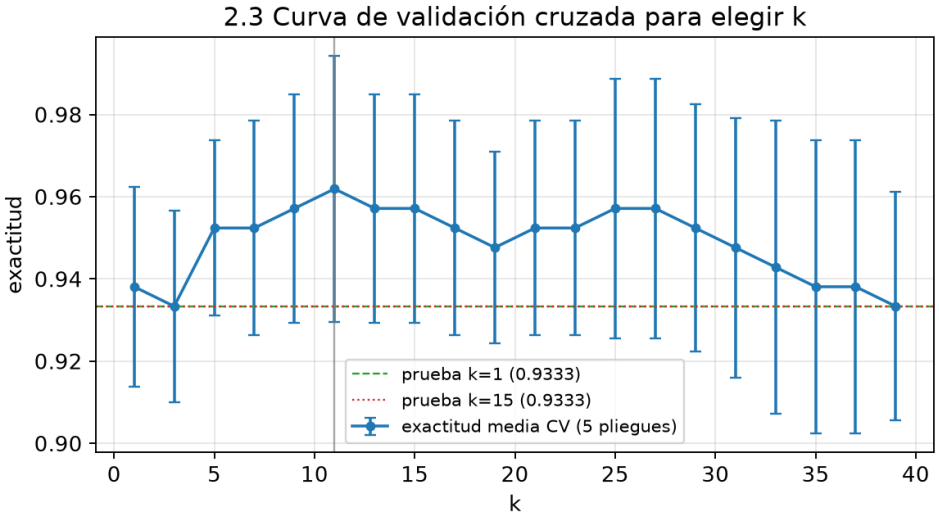


Figura 5: 2.3 Curva de validación cruzada (5 pliegues) sobre el entrenamiento para elegir k; las líneas horizontales marcan la exactitud de prueba de $k = 1$ y $k = 15$ como referencia, no como criterio de selección.

3. Punto 3: evaluación de un clasificador binario

Etiquetas reales y predichas, con la clase 1 como positiva:

$$y = [0, 0, 0, 0, 1, 1, 1, 1, 1, 1], \qquad \hat{y} = [0, 1, 0, 0, 1, 0, 1, 1, 0, 1]. \tag{8}$$

3.1. Matriz de confusión

Filas: clase real; columnas: clase predicha. Recorriendo las diez posiciones:

Cuadro 4: 3.1 Clasificación de cada observación.

#	1	2	3	4	5	6	7	8	9/10
y	0	0	0	0	1	1	1	1	1, 1
$\hat{y}$	0	1	0	0	1	0	1	1	0, 1
Resultado	TN	FP	TN	TN	TP	FN	TP	TP	FN, TP

$$\text{Matriz de confusión} = \begin{array}{c|cc} & \hat{y}=0 & \hat{y}=1 \\ \hline y=0 & TN=3 & FP=1 \\ y=1 & FN=2 & TP=4 \end{array} \quad \Longrightarrow \quad \begin{array}{ll} TP=4, & TN=3, \\ FP=1, & FN=2. \end{array} \tag{9}$$

Comprobación: $TP + TN + FP + FN = 10$ observaciones; positivos reales $TP + FN = 6$; negativos reales $TN + FP = 4$.

3.2. Métricas

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} = \frac{4+3}{10} = 0.70 \ (70\ \%), \tag{10}$$

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{4}{4+1} = 0.80 \ (80\ \%), \tag{11}$$

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{4}{4+2} = \frac{2}{3} \approx 0.6667 \ (66.67\ \%), \tag{12}$$

$$F_1 = \frac{2\,\text{Precision}\,\text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2 \cdot 0.8 \cdot \frac{2}{3}}{0.8 + \frac{2}{3}} = \frac{16/15}{22/15} = \frac{8}{11} \approx 0.7273 \ (72.73\ \%). \tag{13}$$

Qué evalúa cada medida.

- **Accuracy**: proporción de aciertos sobre el total, contando positivos y negativos. Medida global y simétrica; sólo es informativa si las clases están balanceadas y ambos errores cuestan lo mismo.
- **Precision** (valor predictivo positivo): de todo lo que el modelo marcó como positivo, qué fracción lo era realmente. Cuantifica el costo de las *falsas alarmas* (FP); importa cuando un positivo predicho dispara una acción costosa (bloquear una cuenta, notificar a un cliente, iniciar un tratamiento invasivo).
- **Recall** (sensibilidad o exhaustividad): de todos los positivos reales, qué fracción se detectó. Cuantifica el costo de los *falsos negativos* (FN); importa cuando dejar pasar un positivo es grave (enfermedad, fraude, falla de un equipo).
- $\boldsymbol{F_1}$: media armónica de precision y recall. Castiga el desequilibrio entre ambas: un modelo con precision 1.0 y recall 0.1 tiene $F_1 \approx 0.18$ aunque su «mitad» de desempeño sea perfecta. Útil con desbalance de clases cuando ambos tipos de error importan.

3.3. Verificación en Python y análisis

La verificación con `scikit-learn` reproduce exactamente los valores manuales (Tabla 5); el cuaderno incluye además aserciones (`assert`) que fallan si el cálculo manual y el de la biblioteca difieren.

Cuadro 5: 3.3 Cálculo manual frente a `scikit-learn`.

Medida	Manual	`scikit-learn`
`confusion_matrix`	[3, 1; 2, 4]	[3, 1; 2, 4]
`accuracy_score`	0.7000	0.7000
`precision_score`	0.8000	0.8000
`recall_score`	0.6667	0.6667
`f1_score`	0.7273	0.7273

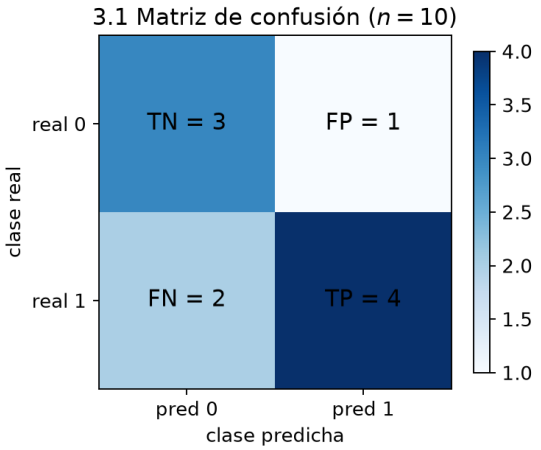


Figura 6: 3.1 Matriz de confusión del ejemplo (filas: clase real; columnas: clase predicha).

Dos situaciones en las que la exactitud por sí sola engaña.

1. **Clases desbalanceadas.** Si el 95 % de los casos son negativos, un clasificador trivial que siempre responde «negativo» obtiene accuracy $= 0.95$ con recall $= 0$ para la clase positiva: nunca detecta un solo positivo. La exactitud es altísima y el modelo es inútil para el objetivo. En el cuaderno se ilustra con `DummyClassifier(strategy="most_frequent")` sobre 95 negativos y 5 positivos: accuracy 0.95, precision 0, recall 0 y $F_1 = 0$.

2. **Costos asimétricos de los errores.** Un FP y un FN rara vez cuestan lo mismo. En tamizaje médico o detección de fraude, un FN puede ser un daño grave mientras que un FP sólo implica una verificación adicional; dos modelos con exactitud idéntica pueden diferir radicalmente en recall y precision, y la exactitud no distingue cuál de los dos errores se está cometiendo. El ejemplo del punto 3 lo muestra: accuracy 0.70 oculta que se dejan pasar 2 de 6 positivos (recall 66.67 %) y que 1 de cada 5 positivos predichos es falso (FP).

Otras dos situaciones, por completitud: conjuntos muy pequeños, donde una sola observación mueve varios puntos porcentuales de exactitud, y problemas multiclase, donde la exactitud global promedia clases con desempeños muy distintos y hace falta la matriz de confusión (o métricas por clase) para verlo.

4. Conclusiones

1. **Frontera lineal.** La regla $\hat{y} = \operatorname{sign}(\mathbf{w}^\top \mathbf{x} + b)$ usa el signo del puntaje, y $\mathbf{w}$ es normal a la recta de decisión. Con $\mathbf{w} = (1, 2)^\top$, $b = -3$ la frontera $x_2 = 1.5 - 0.5x_1$ asigna -1 a $x_1 = (0, 0)$ y $x_2 = (1, 0)$, y $+1$ a $x_3 = (2, 1)$ y $x_4 = (3, 1)$.

2. **Perceptrón.** La actualización sobre una observación mal clasificada incrementa su margen funcional en $\eta(\|\mathbf{x}_i\|_2^2 + 1) > 0$, con independencia del error previo; el margen geométrico no

es necesariamente monótono porque $\|\mathbf{w}\|$ también cambia. En el ejercicio numérico ($\mathbf{w} = 0$, $b = 0$, $\eta = 0.5$, $\mathbf{x} = (2, 1)$, $y = +1$) sí hay error, y resultan $\mathbf{w}_{\text{nuevo}} = (1, 0.5)^\top$, $b_{\text{nuevo}} = 0.5$ y $m_{\text{nuevo}} = 3$, consistente con la fórmula de 1.2.

3. **k-NN.** El orden de cercanía a $q = (1, 1)$ es $x_5 \prec x_2 \prec x_1 \prec x_3 \prec x_4$ y la predicción pasa de **B** ($k = 1$) a **A** ($k = 3$) y de nuevo a **B** ($k = 5$): k controla el compromiso sesgo–varianza. Sobre `make_moons` ($N = 300$, `noise=0.20`) las exactitudes de prueba son 0.9333 ($k = 1$), 0.9556 ($k = 5$) y 0.9333 ($k = 15$); $k = 1$ memoriza el entrenamiento (accuracy 1.0000) con frontera irregular y $k = 5$ generaliza mejor. La selección formal de k se hace por validación cruzada de 5 pliegues sobre el entrenamiento, que aquí escoge $k = 11$ (CV 0.9619, prueba 0.9667), nunca mirando el conjunto de prueba.

4. **Evaluación.** Para el ejemplo del punto 3, $TP = 4$, $TN = 3$, $FP = 1$, $FN = 2$; accuracy 0.70, precision 0.80, recall 0.6667 y $F_1 = 8/11 \approx 0.7273$, verificados con `scikit-learn`. La exactitud sola engaña con clases desbalanceadas o con costos asimétricos de los errores; precision, recall, F_1 y la matriz de confusión son imprescindibles para interpretar el desempeño.

Reproducibilidad

El cuaderno `solucion_taller.ipynb` es autocontenido: no lee archivos externos, genera sus propios datos y sólo requiere `numpy`, `scikit-learn` y `matplotlib`.

Google Colab (recomendado). Subir el archivo a Drive (o *Archivo* → *Subir cuaderno*) y ejecutar *Entorno de ejecución* → *Ejecutar todo*. Colab ya trae las tres bibliotecas; la primera celda de código las comprueba y, si faltara alguna, la instala con `pip`. Las figuras se escriben en `figuras/` y el resumen numérico en `resultados.json` dentro del directorio de trabajo de la sesión (`/content` en Colab), sin afectar la reproducibilidad de los resultados.

Local. Python 3.10 o superior con las tres bibliotecas:

```
python3 -m venv .venv
.venv/bin/python -m pip install numpy scikit-learn matplotlib jupyter
.venv/bin/jupyter notebook solucion_taller.ipynb   # Ejecutar todo
```

El cuaderno fija `np.random.seed(42)` y usa `random_state=42` en `make_moons`, `train_test_split` y `StratifiedKFold`; dos ejecuciones completas producen exactamente los mismos resultados. Versiones de desarrollo: Python 3.14, `numpy 2.5.4`, `scikit-learn 1.9.1`, `matplotlib 3.11.2` (el cuaderno imprime las versiones efectivas al iniciar).

A. Código de las partes implementadas manualmente

Perceptrón: predicción y actualización (1.3)

```
def predecir(w, b, x):
    return 1 if w @ x + b > 0 else -1

def actualizar(w, b, x, y, eta):
    return w + eta * y * x, b + eta * y

w, b, eta, x, y = np.zeros(2), 0.0, 0.5, np.array([2.0, 1.0]), 1.0
if y * (w @ x + b) <= 0:                    # hay error -> se actualiza
    w, b = actualizar(w, b, x, y, eta)
    print(w, b, y * (w @ x + b))            # [1.  0.5] 0.5 3.0
```

k-NN con votación mayoritaria manual (2.1)

```python
def knn_manual(q, X, clases, k):
    d = np.linalg.norm(X - q, axis=1)       # distancia euclidiana
    idx = np.argsort(d, kind="stable")[:k]  # k vecinos mas cercanos
    votos = {}
    for c in clases[idx]:
        votos[str(c)] = votos.get(str(c), 0) + 1
    return sorted(votos.items(), key=lambda kv: (-kv[1], kv[0]))[0][0]
```

Protocolo experimental de 2.2

```python
X, y = make_moons(n_samples=300, noise=0.20, random_state=42)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.30, stratify=y, random_state=42)
for k in (1, 5, 15):
    m = KNeighborsClassifier(n_neighbors=k).fit(X_tr, y_tr)
    print(k, accuracy_score(y_tr, m.predict(X_tr)),
             accuracy_score(y_te, m.predict(X_te)))

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
for k in range(1, 41, 2):
    print(k, cross_val_score(KNeighborsClassifier(n_neighbors=k),
                             X_tr, y_tr, cv=cv).mean())
```

Métricas: cálculo manual y verificación (3)

```python
TP = int(np.sum((y == 1) & (yhat == 1)))
TN = int(np.sum((y == 0) & (yhat == 0)))
FP = int(np.sum((y == 0) & (yhat == 1)))
FN = int(np.sum((y == 1) & (yhat == 0)))

acc = (TP + TN) / (TP + TN + FP + FN)          # 0.70
prec = TP / (TP + FP)                          # 0.80
rec = TP / (TP + FN)                           # 0.6667
f1 = 2 * prec * rec / (prec + rec)             # 0.7273

assert np.array_equal(np.array([[TN, FP], [FN, TP]]),
                      confusion_matrix(y, yhat, labels=[0, 1]))
```